xStore

Business & Module Process Reference

How every part of the app works, how data flows through it, and how the modules connect

Prepared for the Development, Marketing, and Customer Service teams, and for xStore stakeholders

Egypt-based multi-vendor marketplace · Cash-on-Delivery · Currency: EGP

Grounded in the current xStore Flutter codebase and engineering state as of 9 August 2026

Contents

This document is written with Word/Google-Docs heading styles throughout — open the Navigation Pane (Word: View → Navigation Pane) or the document outline (Google Docs: View → Outline) for live, clickable, page-numbered navigation. The list below is a map of what's inside.

How to Read This Document

Each of the following Parts covers one business area of the xStore app. Every Part follows the same shape: a plain-language Business Summary anyone can follow, a Why It Matters note aimed at the business impact, a Data Flow diagram showing step by step what happens, a Connects With table naming every other module it depends on or feeds, and a Key Points list flagging what the team should know — open risks, deliberate design decisions, and gaps.

Diagrams use one visual language throughout the document, so once you've read one, you can read all of them:

Dark pill	Who starts this flow — a Buyer, Vendor, Courier, or Admin.
Grey box	A step the app or a person performs.
Amber diamond	A decision point — the flow branches based on the answer.
Blue box	Data is read, written, or a state changes (e.g. an order status flips).
Purple box	A call to a backend API — the app is talking to a server.
Green pill	A successful end point of the flow.
Red / rose box	A failure, cancellation, or blocked path.
Dashed grey box	A link out to another module — see that module's own Part for detail.

Status labels

- Complete — built and working end to end.
- Partial — the screen and flow are built; some or all of it still runs on mock data, or a real backend endpoint is missing or unstable.
- Pilot — a deliberately small-scale live test of a new capability (currently: the courier delivery pilot).
- Stubbed — UI exists but the data behind it is entirely simulated; no real backend connection.

Business Overview

What xStore Is

xStore is a multi-vendor marketplace app for Egypt. Independent vendors open a store inside the app and list products; buyers browse, order, and pay in cash on delivery (COD) — the dominant and, at launch, the only real payment method. Everything is priced in Egyptian Pounds (EGP).

Who Uses It

- Buyer (Consumer) — browses, orders, and pays cash on delivery.
- Vendor — runs a store, lists products, and fulfils orders (self-delivery or via an xStore courier).
- Courier — an xStore-employed delivery person (piloted, not self-registered) who picks up confirmed orders and standalone package requests, delivers them, and collects cash from the buyer.
- Admin — the xStore team, operating the separate Admin Dashboard: assigns couriers, prices package deliveries, and (in future) manages merchandising and commission configuration.

How xStore Makes Money

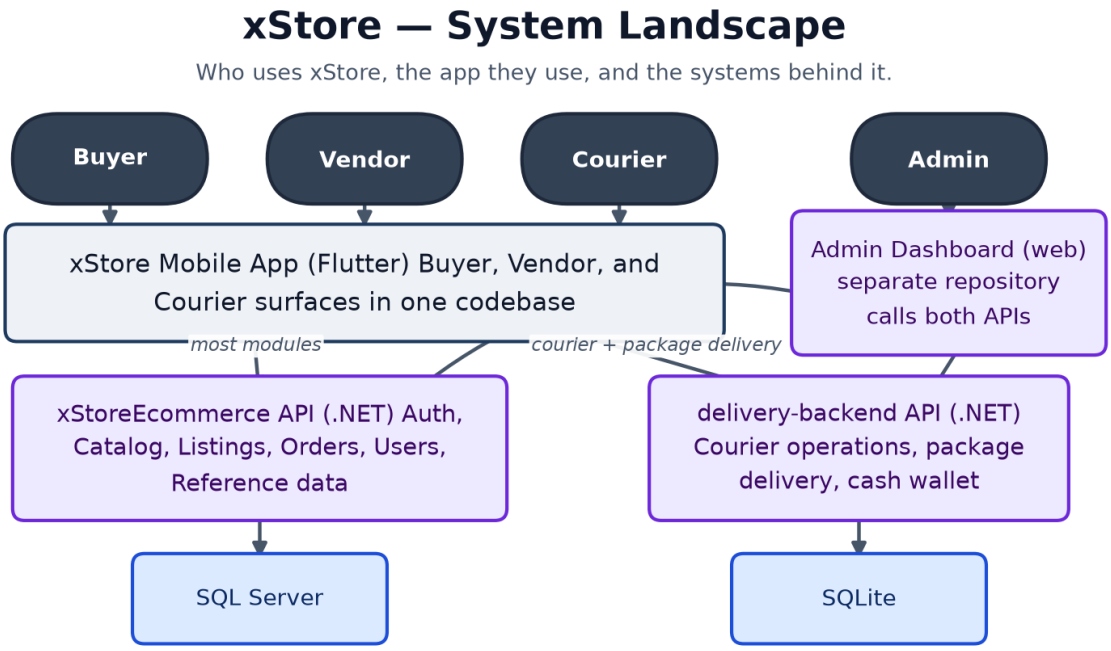
On every sale, xStore keeps a platform commission — a percentage deducted from what the vendor earns, never added on top of what the buyer pays. Buyers always see one clean price. Because xStore is cash-on-delivery, the app itself never holds a buyer's money: either the vendor collects the cash and owes xStore its commission afterward (see Part 10, Commission & Vendor Wallet), or an xStore courier collects the cash directly and the platform holds it from the moment of delivery (see Part 11, Delivery — Courier Operations). Both models are live design decisions, not placeholders — they reflect how a COD-first market like Egypt actually moves money.

Where the App Is Today

Engineering has built the mobile app feature-first: almost every screen a buyer, vendor, or courier needs exists and works against realistic mock data. The main backend (xStoreEcommerce API) is real and live for several modules — authentication, catalog browsing, and reference data — but key pieces are still missing or unstable, most importantly a real Orders/Checkout backend and push notifications. The courier delivery and package-sending features are a deliberate, small pilot on a second, independent backend. The Consolidated Launch-Readiness & Risk Register near the end of this document rolls up every open item across modules into one prioritized list for planning.

System Landscape

One Flutter mobile app serves all three field roles — Buyer, Vendor, and Courier — from a single codebase, switching the screens and navigation shown based on the signed-in account's role. It talks to two independent backend systems, and a separate web-based Admin Dashboard talks to both of the same backends.

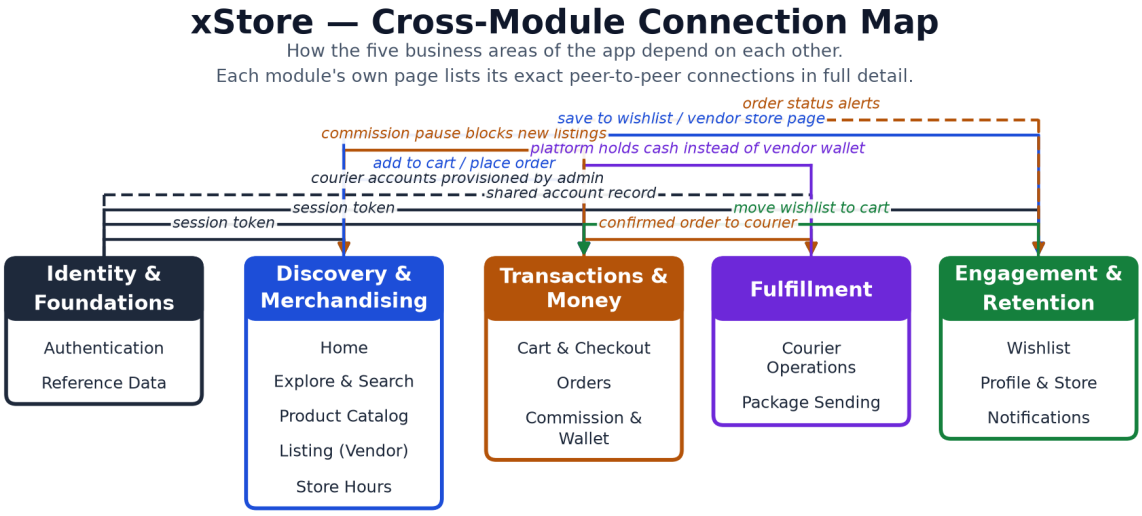


Two independent backends, two independent auth schemes — the mobile app and the admin dashboard both call both of them. Most modules use xStoreEcommerce; only the delivery features (courier pick-up/deliver, package sending, cash wallet) use delivery-backend.

Most modules in this document — Authentication, Catalog, Listings, Cart, Orders, Users, Reference Data — run against the xStoreEcommerce API. Only the delivery features (courier pick-up/deliver, package sending, and the courier cash wallet) run against the separate delivery-backend. The two systems have separate authentication schemes — worth keeping in mind for anyone debugging a login or permissions issue: which backend a report actually points to is not always obvious from the app screen alone.

How the Modules Connect

The map below groups the app's 15 modules into five business areas and shows how those areas depend on each other — the shape of the system at a glance. Each module's own Part later in this document lists its exact module-to-module connections in full detail.



Solid arrows = live data flow today. Dashed arrows = the connection is designed but not yet built (Notifications has no real push infrastructure yet — see Part 15).

Module Index

Part	Module	Status	Actors
1	**Authentication & Identity**	Partial	Buyer, Vendor, Courier
2	**Reference Data & Location Taxonomy**	Partial	Buyer, Vendor, Admin
3	**Home (Merchandising Feed)**	Partial	Buyer
4	**Explore & Search**	Partial	Buyer
5	**Product Catalog & Detail**	Partial	Buyer
6	**Listing (Vendor Product Management)**	Partial	Vendor
7	**Store Hours**	Partial	Vendor, Buyer
8	**Cart & Checkout**	Partial	Buyer
9	**Orders & Fulfillment Status**	Partial	Buyer, Vendor, Courier
10	**Commission & Vendor Wallet**	Partial	Vendor, Admin
11	**Delivery — Courier Operations**	Pilot — client-side complete	Courier, Admin
12	**Delivery — Package Sending**	Pilot — client-side complete, fully mocked	Buyer, Vendor, Courier, Admin
13	**Wishlist**	Partial	Buyer
14	**Profile & Vendor Store Page**	Partial	Buyer, Vendor
15	**Notifications**	Stubbed	Buyer, Vendor

PART 1

Authentication & Identity

Status: Partial — Email/password login and registration are wired to the live backend. Phone OTP and Google sign-in are wired to their own live endpoints; Apple and Facebook are hidden (no backend route yet).

Location: lib/features/auth/

Backend: xStoreEcommerce API — POST /api/auth/login, /register, /send-login-otp, /login-with-otp, /google/{consumer|vendor}/login

Actors: Buyer, Vendor, Courier

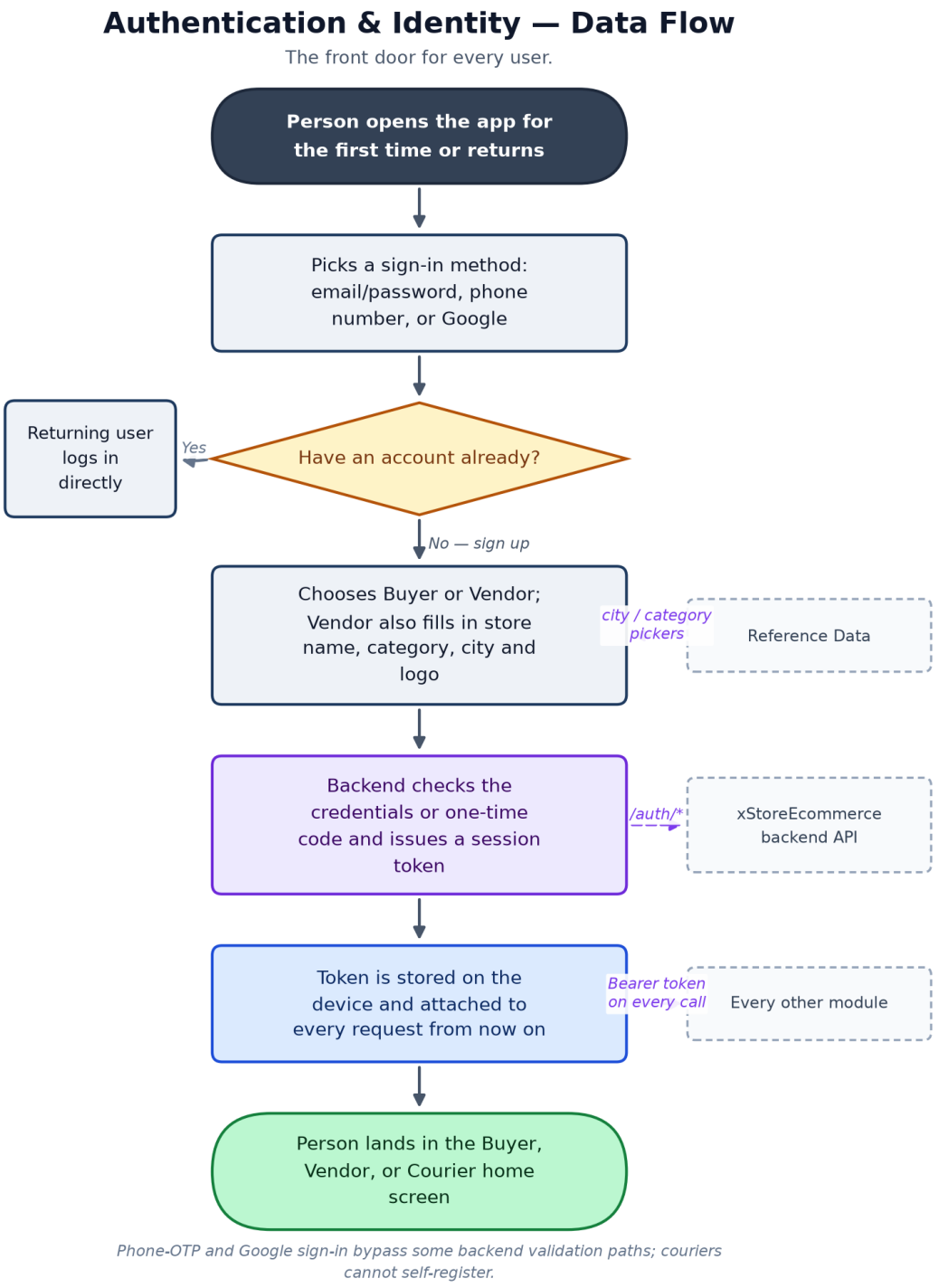
Business Summary

The front door for every user. A new visitor picks a role — Buyer or Vendor — and signs up with email/password, a verified Egyptian mobile number (one-time code), or Google. Vendors add their store profile at the same step. Couriers are the one exception: their accounts are created directly by the xStore team, not through self sign-up, and they sign in through a dedicated delivery-staff screen.

Why It Matters

Because the entire app is login-gated, this is the first thing every customer and every vendor touches — any friction here is lost signups, full stop. It is also the foundation every other module quietly depends on: the session token issued here is what proves who is buying, who is selling, and who is delivering on every later screen.

Data Flow



Phone-OTP and Google sign-in bypass some backend validation paths; couriers cannot self-register.

Connects With

Module	Relationship	Direction
Every module	Supplies the session token every other screen needs to call the	provides

Module	Relationship	Direction
	backend.	
Profile	Profile screen reads and edits the same account record created here.	shared data
Delivery — Courier Operations	Courier accounts are provisioned outside self sign-up and use a separate login screen.	special case
Reference Data & Location Taxonomy	Vendor registration pulls city/store-category lists from this module.	consumes

Key Points for the Team

- Session friction directly costs signups — this is the highest-leverage screen for growth/marketing to monitor drop-off on.
- Phone OTP and Google sign-in currently do not go through the same validation path as email/password — worth a security review before scaling paid acquisition through those channels.
- There is no self-service path to become a courier by design — this is a deliberate trust control (see Delivery), not a gap.
- Logout is currently local-only (the device forgets the token); the backend is never told the session ended.

PART 2

Reference Data & Location Taxonomy

Status: Partial — All four lists read live from the backend; none are yet wired into every screen that should use them (e.g. Listing's category picker still uses a hardcoded list).

Location: lib/features/{catalog_categories, store_categories, cities, governments}/

Backend: xStoreEcommerce API — GET /api/categories, /api/store-categories, /api/cities, /api/governments

Actors: Buyer, Vendor, Admin

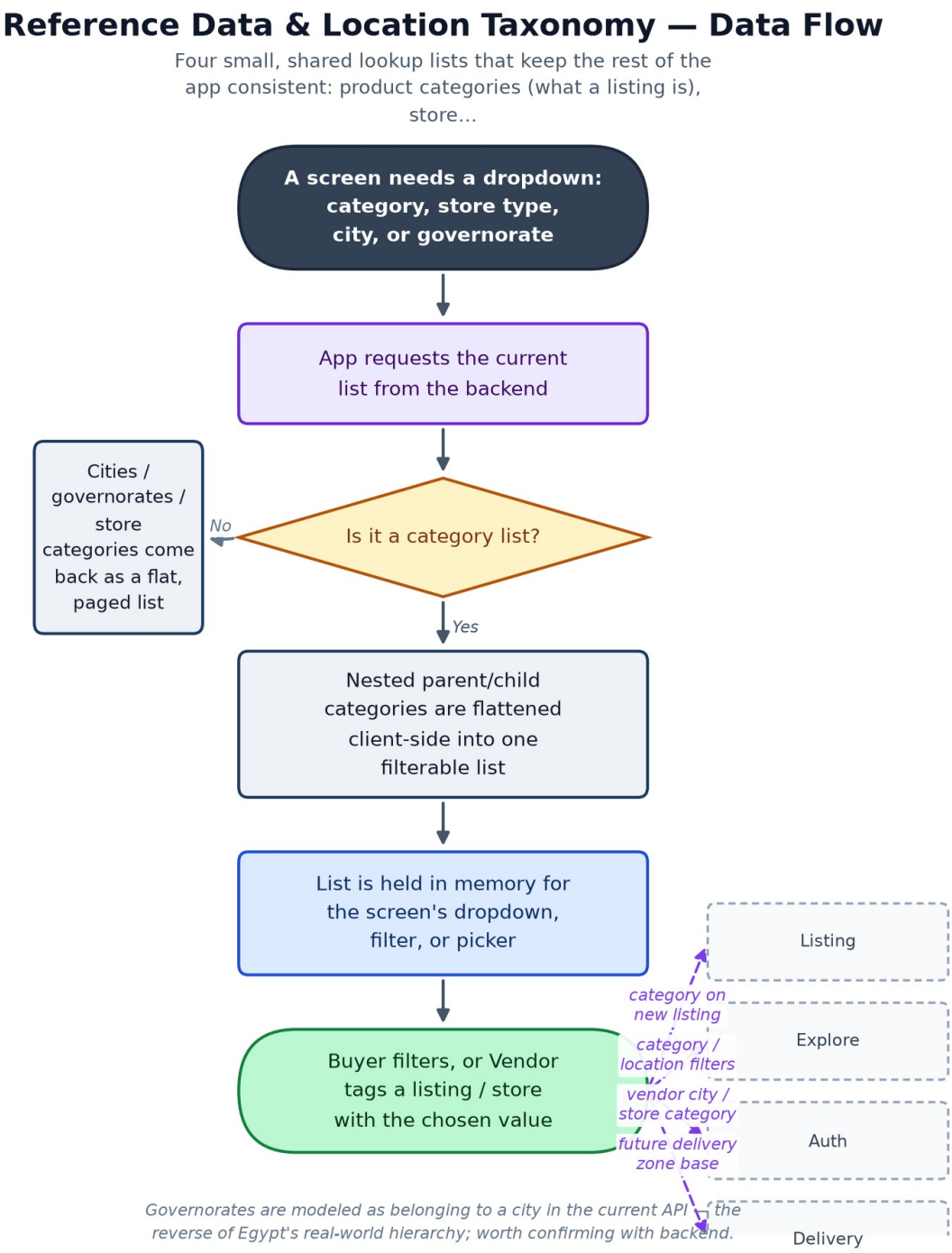
Business Summary

Four small, shared lookup lists that keep the rest of the app consistent: product categories (what a listing is), store categories (what kind of shop a vendor runs), cities, and governorates. Nobody buys or sells here directly — it is the shared vocabulary every other screen with a dropdown or filter draws from.

Why It Matters

Consistent categories and locations are what make search, filtering, and delivery-zone planning possible. If Vendor A calls their category 'Phones' and Vendor B calls it 'Mobiles', buyers lose both listings in search. This module is low-glamour but it is the taxonomy the whole catalog and the future delivery-zone plan are built on.

Data Flow



Connects With

Module	Relationship	Direction
Listing	Product category and vendor store-category pickers should draw from here (Listing's list is still hardcoded — a known gap).	feeds

Module	Relationship	Direction
Explore & Search	Category and location filters.	feeds
Authentication & Identity	Vendor registration's city and store-category fields.	feeds
Delivery	The intended base for future delivery-zone coverage planning.	future dependency

Key Points for the Team

- This is reference data, not a customer-facing feature — but every category mismatch downstream (search, listing forms) traces back here.
- Listing's create-product screen still uses its own hardcoded category list instead of this module — replacing it is a tracked follow-up, not yet done.
- Good module for Admin dashboard investment: whoever can edit these lists controls the taxonomy the whole catalog search depends on.

PART 3

Home (Merchandising Feed)

Status: Partial — Rich UI built; banners and hot-deals sections pull from the backend when available, category shortcuts are still local.

Location: lib/features/home/

Backend: xStoreEcommerce API — GET /api/home/banners, /api/home/hot-deals, /api/home/categories (currently 500s live — see gap register)

Actors: Buyer

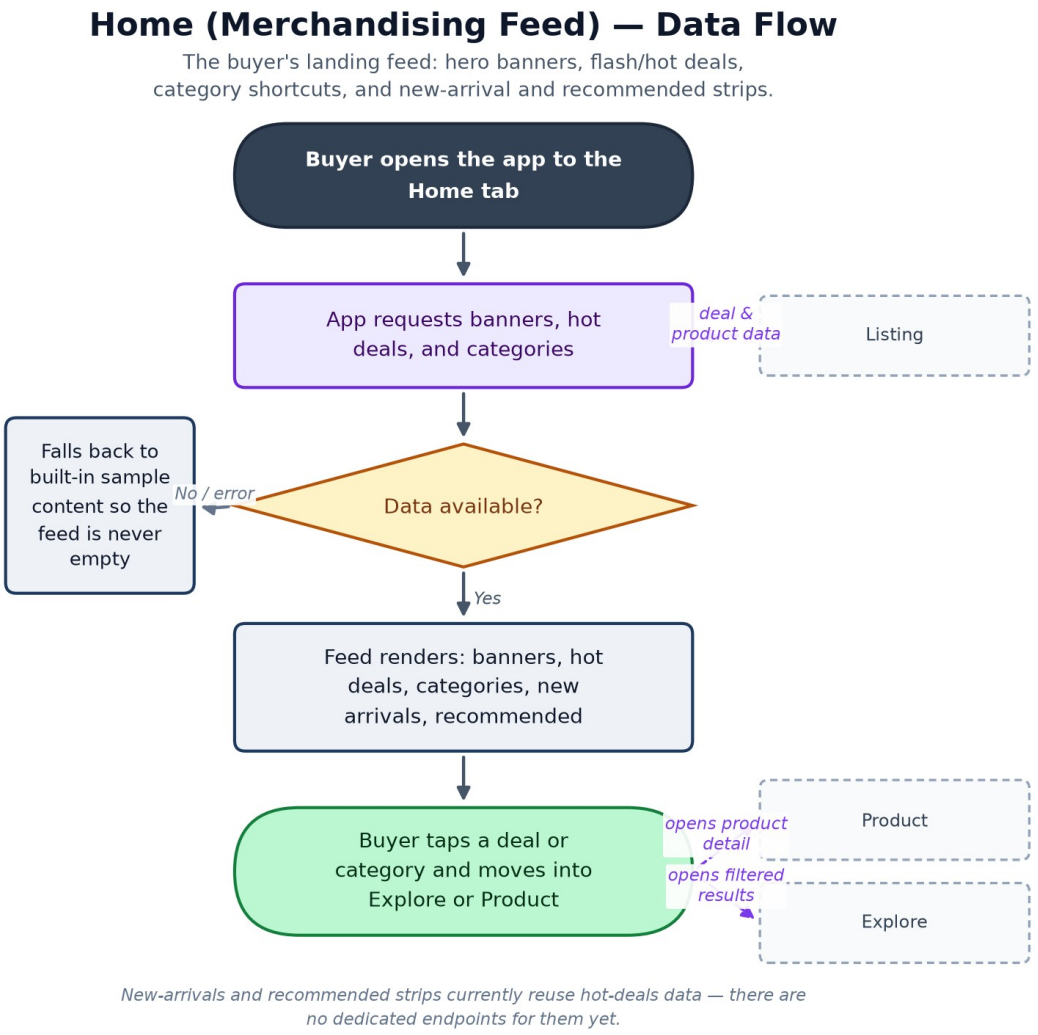
Business Summary

The buyer's landing feed: hero banners, flash/hot deals, category shortcuts, and new-arrival and recommended strips. This is xStore's shop window — whatever is featured here is what buyers see first, before they've searched for anything.

Why It Matters

This is the single most valuable piece of in-app marketing real estate. Today, what's featured is fixed at build time — there is no way for marketing or an admin to change a banner or push a seasonal campaign without a code release, which is a real limitation for running promotions.

Data Flow



Connects With

Module	Relationship	Direction
Listing	Every deal and category tile is a listing created by a vendor.	consumes
Product	Tapping a deal opens that product's detail page.	hands off to
Explore & Search	Category shortcuts open a filtered search.	hands off to

Key Points for the Team

- No merchandising controls exist yet — marketing cannot schedule or swap a banner without an app release. This is a concrete admin-dashboard priority if campaigns are planned.
- The live backend's /api/home endpoint currently 500s under load (see the consolidated gap register) — the app tolerates this by composing the feed from other endpoints instead, so buyers are not currently affected, but it should still be fixed.

PART 4

Explore & Search

Status: Partial — Search, filters, sort, and recent searches are functional against mock or live data. An advanced-filters panel is marked 'coming soon' in the UI.

Location: lib/features/explore/

Backend: xStoreEcommerce API — GET /api/listings (search), /api/listings/suggestions

Actors: Buyer

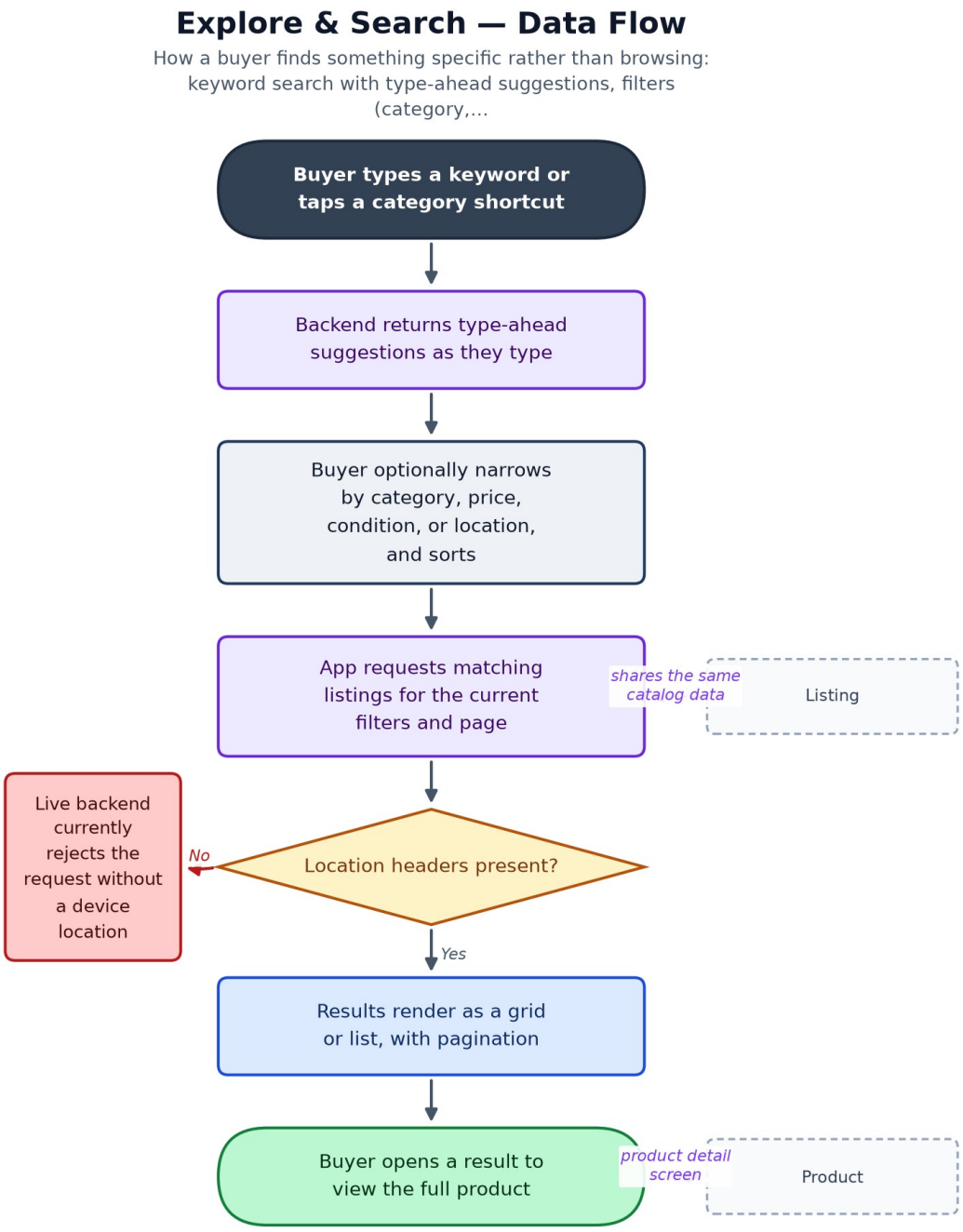
Business Summary

How a buyer finds something specific rather than browsing: keyword search with type-ahead suggestions, filters (category, price, condition, location), sorting, a grid/list toggle, and recent searches.

Why It Matters

Search quality is a direct conversion lever — if a buyer can't find the product they're picturing, they leave rather than keep browsing. This is also the screen most exposed to the current backend defect where listings require a device location to return any results at all (see gap register) — worth customer-service and marketing awareness, since it can look like 'search is broken' or 'no products exist' to a real user.

Data Flow



Explore and Listing's create-product flow both use the same /listings path for different purposes — the backend must tell search from creation apart.

Connects With

Module	Relationship	Direction
Listing	Search results are the same catalog Vendors publish to.	reads

Module	Relationship	Direction
Product	Opening a result goes to the Product module's detail page.	hands off to
Reference Data & Location Taxonomy	Category and location filter options.	consumes

Key Points for the Team

- Live-probed 2026-08-06: the backend now hard-requires X-Latitude/X-Longitude headers and a 50km radius filter the app does not send — this breaks Explore and Home's hot deals for every real user until either the app sends location or backend adds a no-location fallback. This is the single highest-priority live defect for customer service to know about.
- 'More filters — coming soon' is still visible in the UI; treat as a known, intentional gap, not a bug report.

PART 5

Product Catalog & Detail

Status: Partial — Detail screen and the full reviews screen (paginated list, write/edit/delete a review) are both feature-complete against mock/live data.

Location: lib/features/product/

Backend: xStoreEcommerce API — GET /api/listings/{id}, /similar, /reviews; POST/PUT/DELETE /api/listings/{id}/reviews for writing, editing, and deleting a buyer's own review

Actors: Buyer

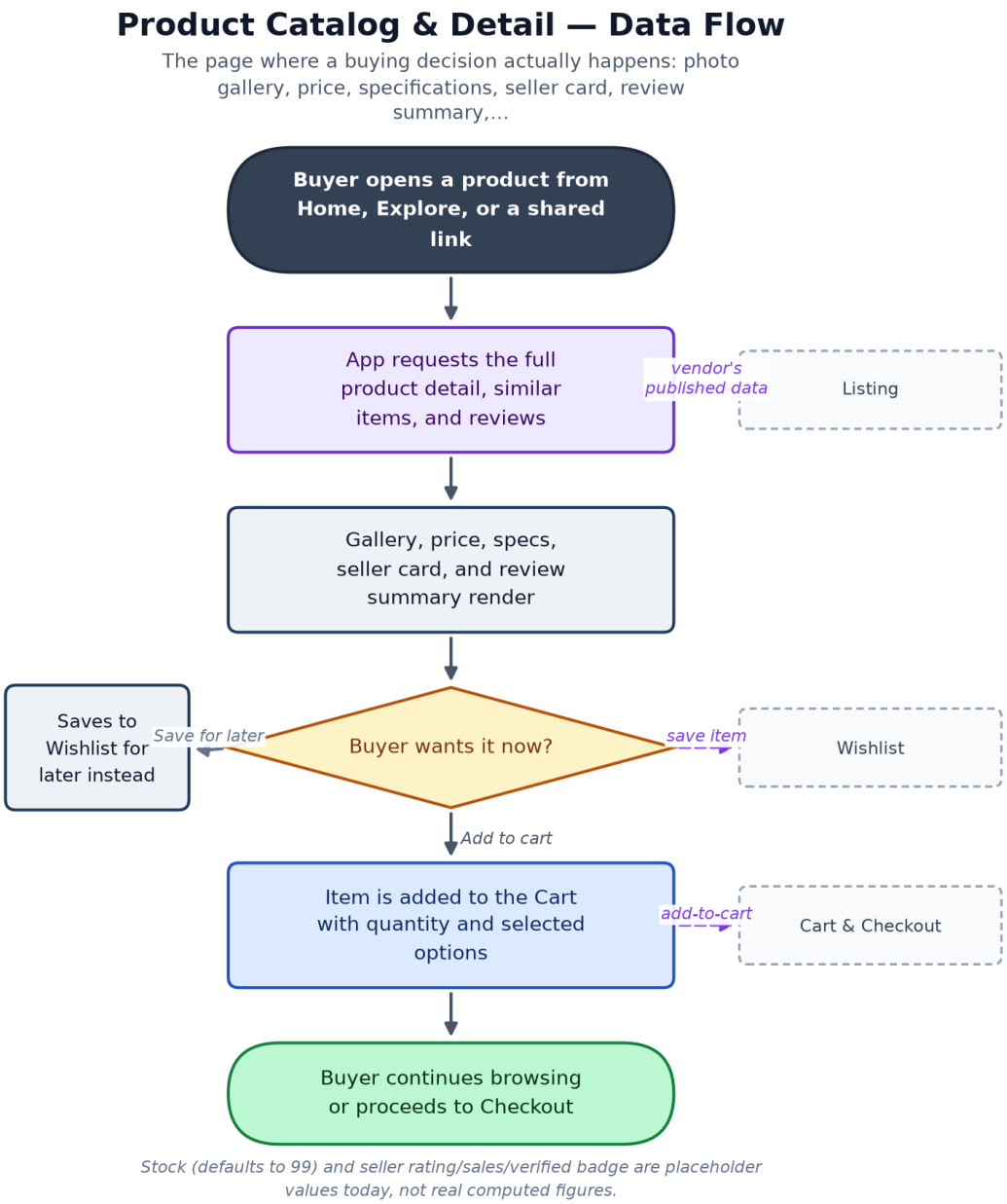
Business Summary

The page where a buying decision actually happens: photo gallery, price, specifications, seller card, review summary, similar products, and add-to-cart.

Why It Matters

This is the richest conversion surface in the app, and it currently shows some numbers that are not real: stock defaults to a flat 99 units, and seller trust signals (4.9 rating, 230 sales, verified badge) are placeholder values, not computed from actual data. Showing fabricated trust and stock numbers to real buyers is a credibility and possibly legal/consumer-protection risk that should be closed before any paid marketing sends traffic here at scale.

Data Flow



Connects With

Module	Relationship	Direction
Listing	Every product page is one vendor listing rendered in detail.	reads
Cart & Checkout	Add-to-cart hands the item to the cart.	feeds
Wishlist	Save-for-later hands the item to the wishlist.	feeds

Key Points for the Team

- Placeholder stock (99) and placeholder seller trust figures (rating/sales/verified) must be replaced with real data before this is used to justify marketing spend — buyers are currently seeing numbers that are not true.
- Reviews are a genuinely complete feature, not a stub: buyers can write, edit, and delete their own review from a paginated full reviews screen — worth marketing knowing this is real and usable today, not a coming-soon item.

PART 6

Listing (Vendor Product Management)

Status: Partial — Form validation and vendor's own listings view are solid; photo upload and true SKU/variant support are not implemented.

Location: lib/features/listing/

Backend: xStoreEcommerce API — POST /api/listings, GET /api/listings/mine, PATCH/DELETE /api/listings/{id}

Actors: Vendor

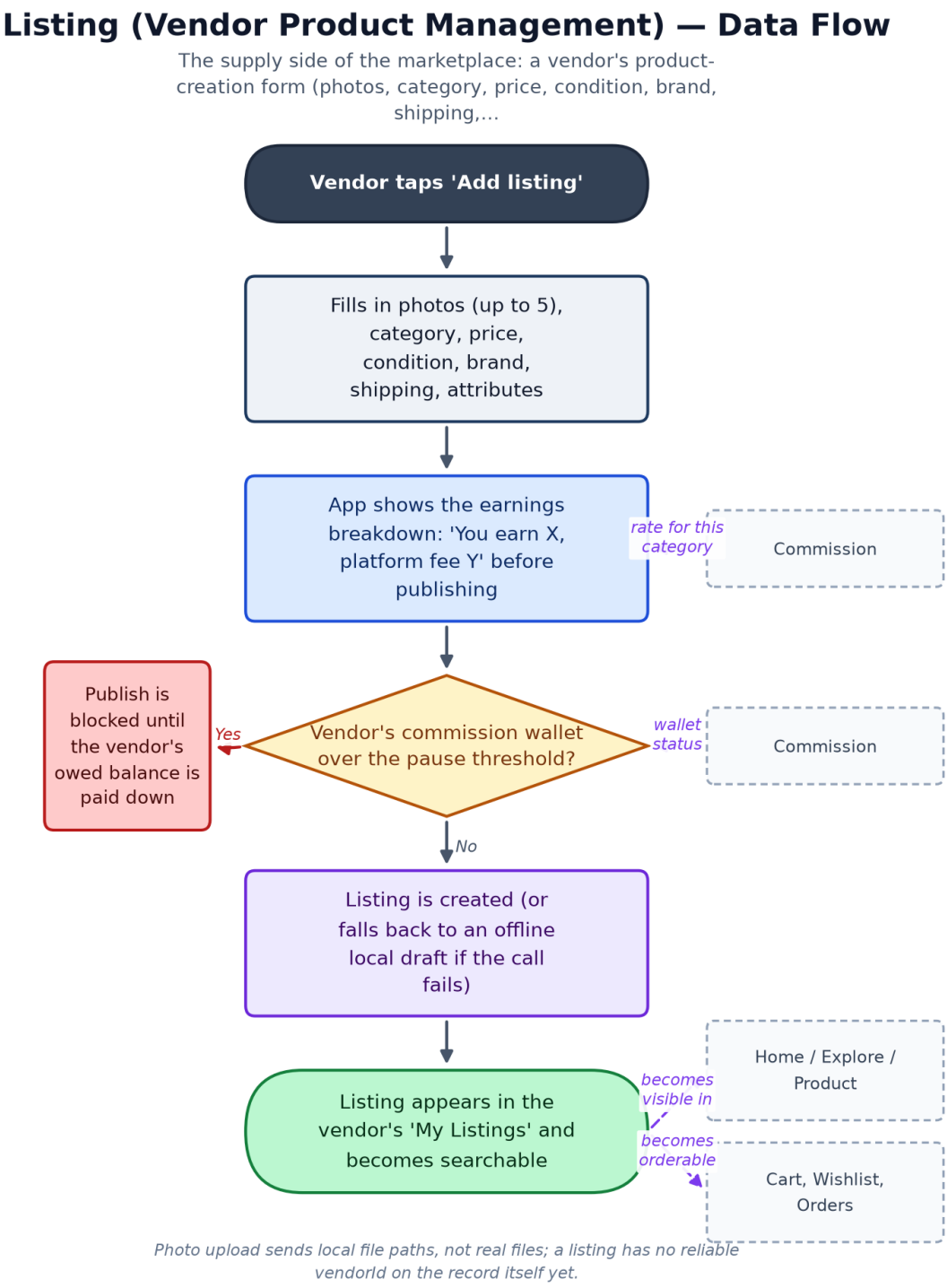
Business Summary

The supply side of the marketplace: a vendor's product-creation form (photos, category, price, condition, brand, shipping, free-form attributes) and a 'my listings' view showing views, saves, and inquiries per product.

Why It Matters

No listings, no marketplace — this is the module that determines whether there is anything to sell at all. Two gaps matter commercially: photos are stored as local file paths rather than actually uploaded anywhere the backend or other buyers can see, and a listing carries no reliable vendor-ownership id on the record itself. Both need to be closed before onboarding real vendors at any volume.

Data Flow



Connects With

Module	Relationship	Direction
Home, Explore, Product	This module's listing record is the backbone every browsing	feeds

Module	Relationship	Direction
	screen displays.	
Cart, Wishlist, Orders	All three operate on the same listing entity.	feeds
Commission & Vendor Wallet	Shows the per-listing fee breakdown and can block publishing when a vendor's balance is over threshold.	reads / is gated by
Reference Data & Location Taxonomy	Should source its category picker from here — currently a separate hardcoded list.	should consume

Key Points for the Team

- Image upload does not actually upload — this is a launch blocker for any real vendor onboarding, since their product photos never leave the device in a usable form.
- No vendorId on the listing record itself — ownership is only inferred from whose token created it, which is fragile for future features like transferring or auditing listings.
- No true variant/SKU model — one price and one stock number per listing, with free-form attributes only. Fine for a simple pilot catalog, a real constraint for vendors selling size/colour variants.
- The commission pause enforcement (built) only blocks publishing new listings — it does not yet auto-pause a vendor's already-live listings when they go over threshold, which is a known enforcement gap.

PART 7

Store Hours

Status: Partial — Validated and functional against mock or live data; low-risk, self-contained.
Location: lib/features/store/
Backend: xStoreEcommerce API — GET/PUT /api/vendors/{id}/store-hours, PATCH /api/vendors/{id}/store-status
Actors: Vendor, Buyer

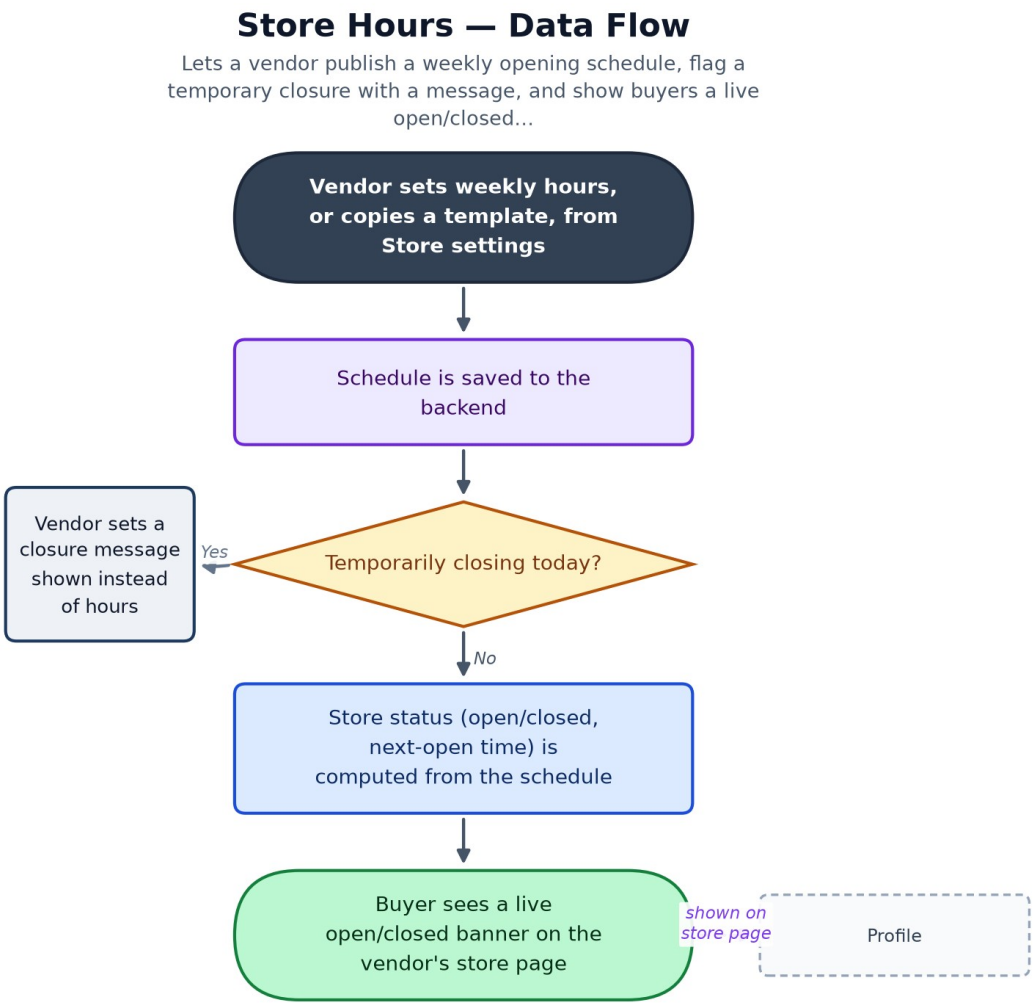
Business Summary

Lets a vendor publish a weekly opening schedule, flag a temporary closure with a message, and show buyers a live open/closed banner on their store page.

Why It Matters

A small but genuine trust signal for a local marketplace — it sets expectations on how fast a buyer's order will be picked up. Low risk and largely done.

Data Flow



Connects With

Module	Relationship	Direction
Profile	The open/closed banner is displayed on the vendor's public store page.	feeds

Key Points for the Team

- Self-contained and low-risk; no significant gaps beyond backend route registration housekeeping.

PART 8

Cart & Checkout

Status: Partial — Full UI (multi-vendor grouping, coupons, 3-step checkout) works against mock/live data; the cart itself is not yet persisted on the backend, and no real orders backend exists to checkout into.

Location: lib/features/cart/

Backend: xStoreEcommerce API — GET/POST/PATCH/DELETE /api/cart/{id}/*, POST /api/cart/coupons/apply — Orders creation is not yet a real backend endpoint (see gap register)

Actors: Buyer

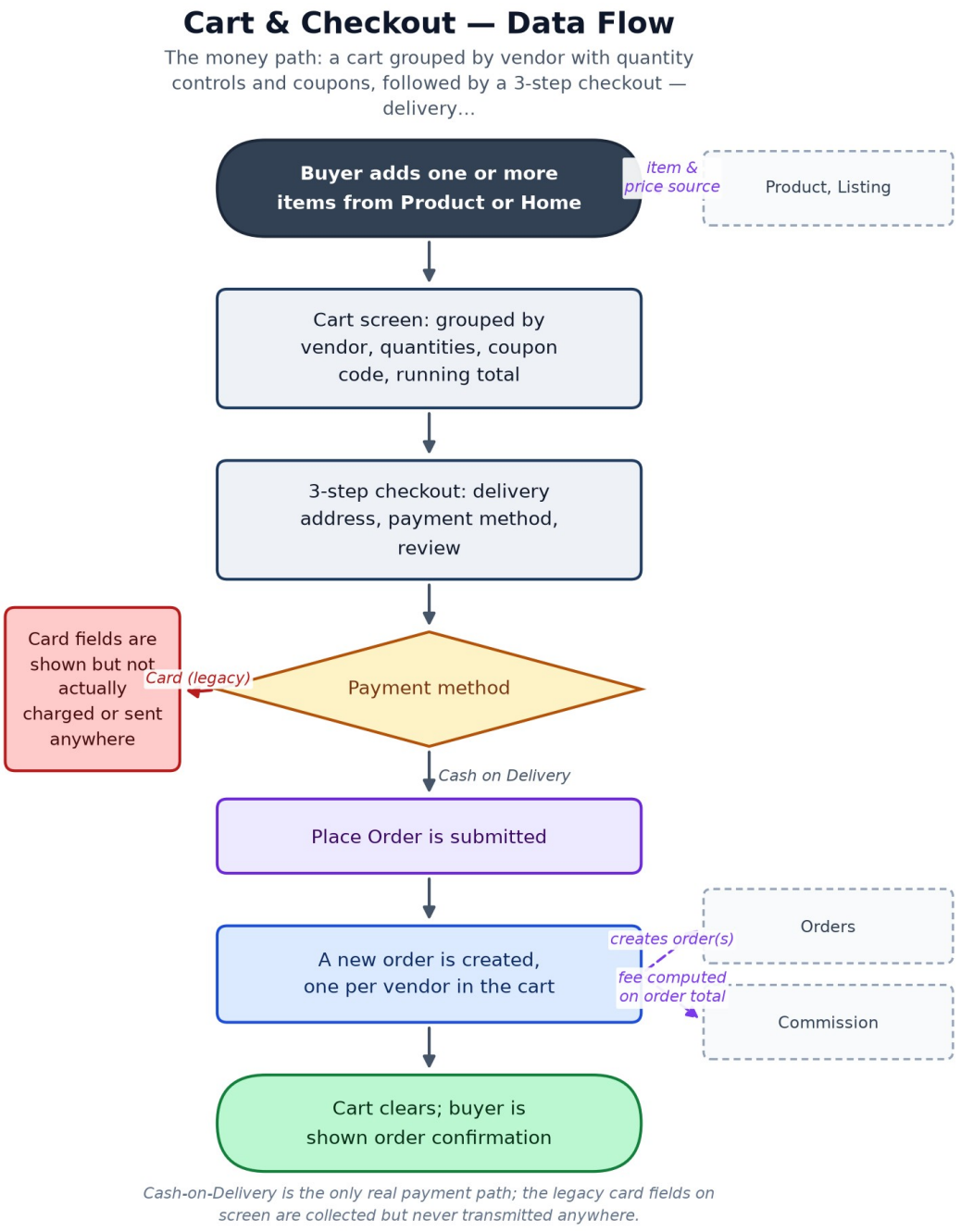
Business Summary

The money path: a cart grouped by vendor with quantity controls and coupons, followed by a 3-step checkout — delivery address, payment, review — ending in Place Order. Vendor accounts are deliberately blocked from using the cart.

Why It Matters

This is the point every other module exists to lead a buyer to. Two things matter for the business right now: xStore is a cash-on-delivery marketplace by design, but the checkout form still shows card-payment fields that are collected on screen and then silently never sent anywhere — those should be removed so buyers aren't asked for card details the app doesn't use. Separately, the cart itself resets if the app restarts while running on mock data, and there is still no real backend to place an order into.

Data Flow



Connects With

Module	Relationship	Direction
Product, Listing	Cart line items are built from listing/product data.	reads
Orders & Fulfillment Status	Placing an order hands the cart contents off to Orders.	creates

Module	Relationship	Direction
Commission & Vendor Wallet	Each vendor's share of the order becomes the base the commission fee is calculated on.	feeds

Key Points for the Team

- Card-payment fields are collected on the checkout form but never sent to any payment processor — since xStore is COD-only, these fields should be removed rather than left as a confusing, non-functional step for buyers.
- The cart is an in-memory list while running on mock data — it does not survive an app restart. Needs a real persisted cart once the backend supports it.
- Checkout currently uses a hardcoded sample delivery address instead of the buyer's saved address from Profile — a real gap once real orders exist.
- There is still no real Orders/Checkout backend endpoint at all (confirmed by a live backend probe) — this is the top structural blocker before Commission or Courier features can run on real order data instead of mocks.

PART 9

Orders & Fulfillment Status

Status: Partial — Full lifecycle UI (buyer + vendor sides) is built and wired to mock/live data patterns, but there is no live Orders backend yet — this module runs entirely on mocked data today.

Location: lib/features/orders/

Backend: Planned: xStoreEcommerce API — GET /api/orders/*, POST /api/orders/{id}/confirm|reject|processing|shipped|delivered|cancel (not yet live — see gap register)

Actors: Buyer, Vendor, Courier

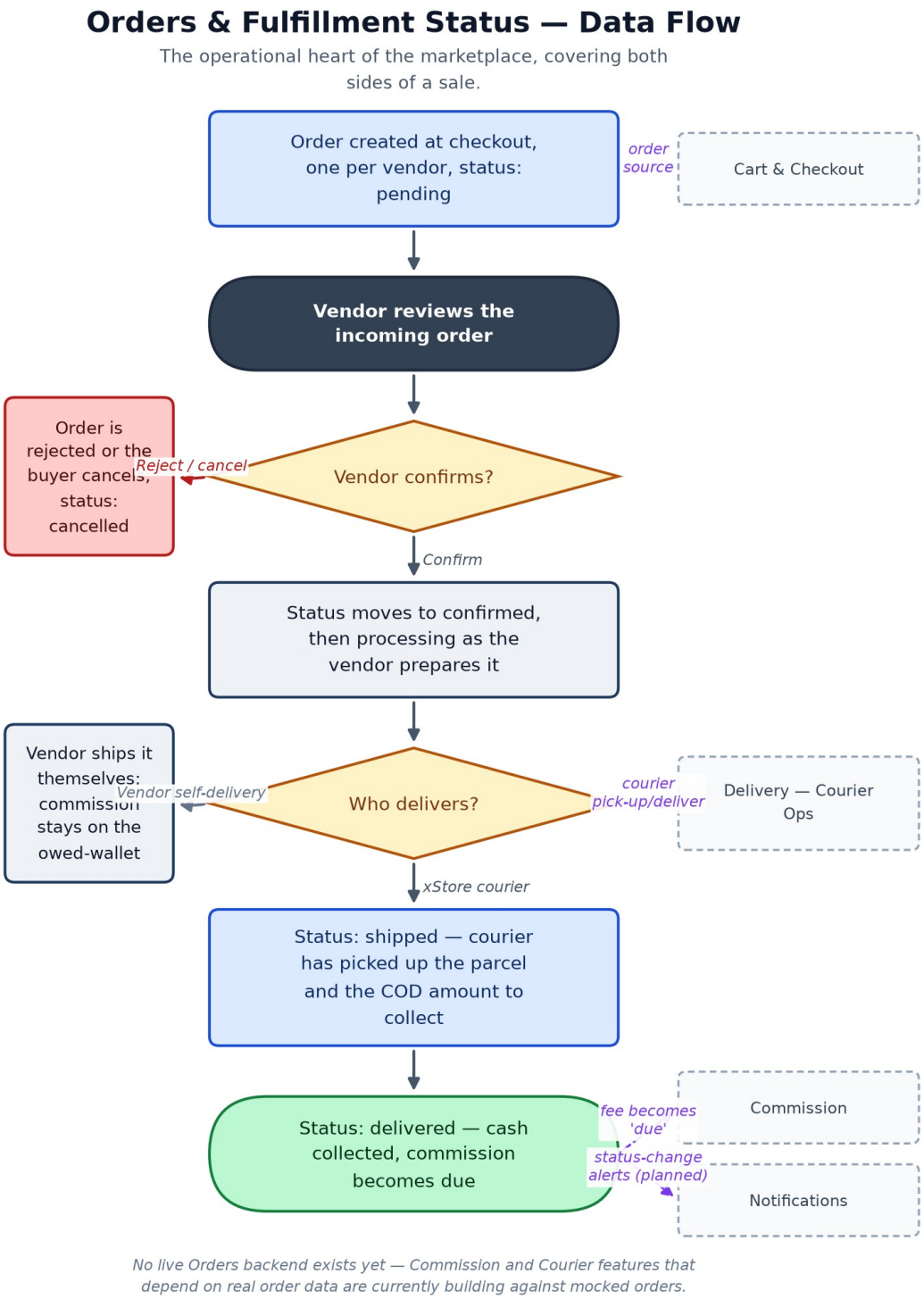
Business Summary

The operational heart of the marketplace, covering both sides of a sale. Buyers see their order list, a status timeline (pending → confirmed → shipped → delivered), and can cancel or reorder. Vendors see incoming orders and move each one through confirm, reject, processing, shipped, and delivered.

Why It Matters

Every commission calculation, every courier assignment, and most customer-service conversations trace back to an order's status here. One design choice is worth calling out to engineering and CS both: an order item stores a frozen snapshot of the product (name, image, price) at the time of purchase, so a vendor editing or deleting a product later never corrupts historical order records — that's a deliberate, good decision, not an oversight.

Data Flow



No live Orders backend exists yet — Commission and Courier features that depend on real order data are currently building against mocked orders.

Connects With

Module	Relationship	Direction
Cart & Checkout	Every order originates from a placed cart.	created by
Commission & Vendor Wallet	Delivery is the trigger that turns a commission fee from 'pending' to 'due'.	drives
Delivery — Courier Operations	Courier pick-up/drop-off reuses this module's own order-status values.	shares state model with
Notifications	Every status change is a natural trigger for a buyer/vendor alert — not yet wired since push does not exist.	should feed

Key Points for the Team

- No real Orders backend exists yet — this is the single biggest structural dependency blocking Commission, Courier Operations, and Notifications from moving off mock data. Flag as the top engineering priority underneath the visible feature work.
- The payment-method list still includes non-Egyptian options left over from an earlier market; only Cash-on-Delivery should remain for the Egypt launch.
- Order-item snapshotting (freezing product name/price/image at purchase time) is a deliberate, correct design — call this out positively in any backend handoff, it should be preserved.
- There is no courier/tracking integration on the buyer-facing side yet ('tracking — coming soon').

PART 10

Commission & Vendor Wallet

Status: Partial — Client-side breakdown, wallet balance, and warn/pause enforcement are built and unit-tested. Rates and thresholds live as constants in the app today — there is no backend commission-config endpoint yet, so they cannot be changed without a release.

Location: lib/features/commission/

Backend: No dedicated backend endpoint yet — computed client-side from order totals; rate is a single flat starter percentage

Actors: Vendor, Admin

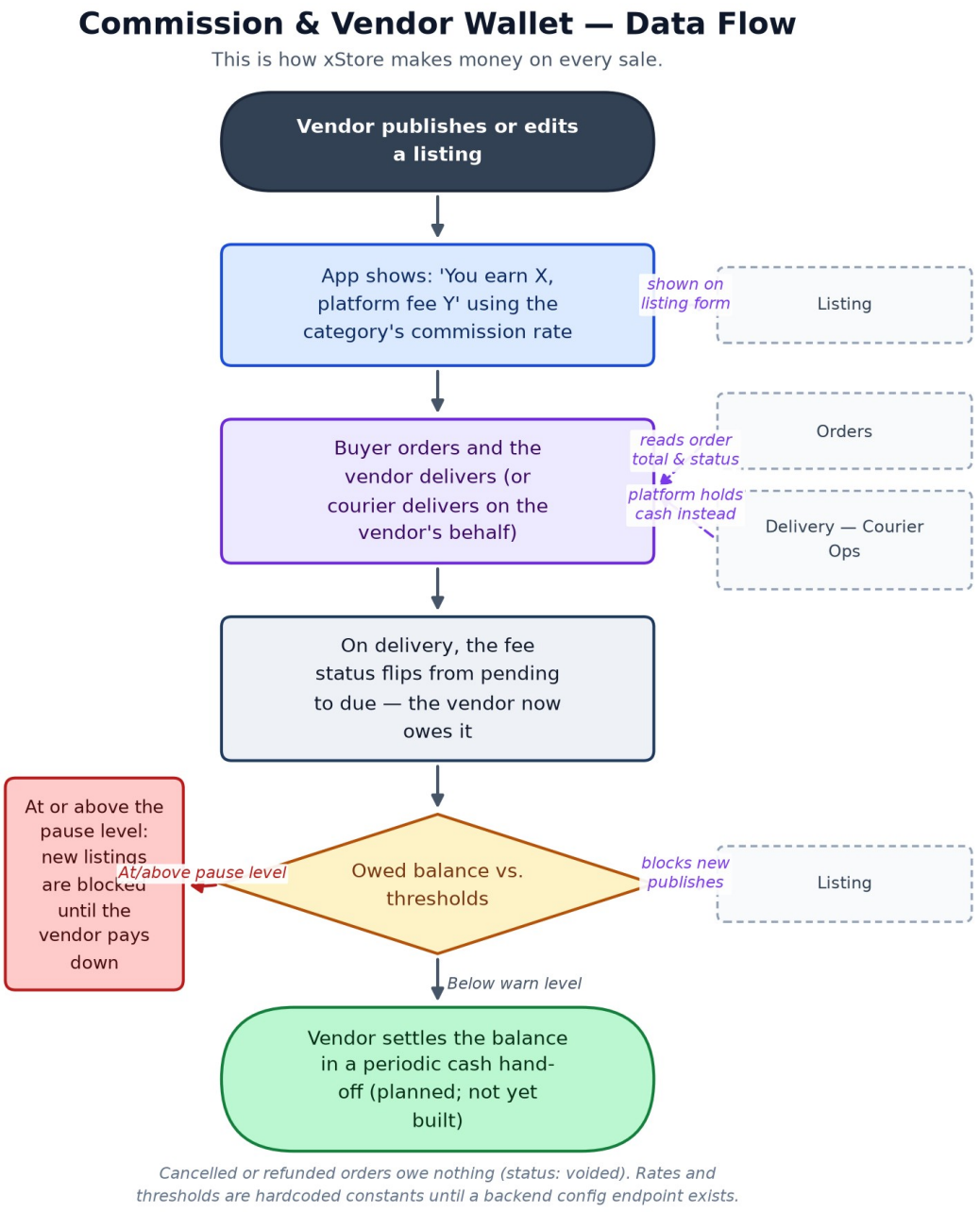
Business Summary

This is how xStore makes money on every sale. A flat starter percentage (2% at launch, configurable per category later) is deducted from what the vendor earns — never added on top of what the buyer pays. The buyer always sees one clean price. The vendor sees the fee and can watch how much they owe the platform build up in a running wallet balance.

Why It Matters

Because xStore is cash-on-delivery, the app never actually holds the buyer's money — the vendor (or courier, see Delivery) collects cash directly. That makes the vendor's commission a debt owed to xStore rather than money xStore can simply deduct. Two owner-set thresholds manage the risk of that debt growing unchecked: a warn level that nudges the vendor to pay up, and a pause level that blocks the vendor from publishing new listings until they do.

Data Flow



Connects With

Module	Relationship	Direction
Listing	Shows the earn/fee breakdown at publish time and can block new listings when a vendor is over the pause threshold.	reads / gates
Orders & Fulfillment Status	Delivery is what turns a fee from pending to due; cancel/refund voids it.	reads

Module	Relationship	Direction
Delivery — Courier Operations	When xStore's own courier delivers an order, the platform holds the cash directly instead of relying on the vendor owed-balance model.	alternate money flow for

Key Points for the Team

- Rates (currently a flat 2% starter rate) and both thresholds (warn: 100 EGP, pause: 200 EGP) are hardcoded in the app, not configurable from any admin screen — every rate or threshold change today requires an app release, which matters for how fast the business can react on pricing.
- There is no settlement/hand-off flow yet — the wallet shows a live owed balance, but there is no way in the app to mark it as paid once a vendor settles up in person.
- Enforcement only blocks a vendor from publishing new listings when over threshold — it does not auto-pause their existing already-live listings, so an over-threshold vendor can keep selling what's already posted.
- Because this module computes revenue client-side, a technically savvy vendor could in principle see how the calculation works — real rates and balances should move server-side before this is trusted at scale.

PART 11

Delivery — Courier Operations

Status: Pilot — client-side complete — Full pick-up → deliver → cash flow is built and emulator-verified end-to-end. Runs against a separate, standalone delivery backend (not the main marketplace API) when live.

Location: lib/features/delivery/ (courier side)

Backend: Standalone delivery-backend (.NET) — /api/courier/*, /api/admin/couriers/* — separate service, separate JWT from the main marketplace API

Actors: Courier, Admin

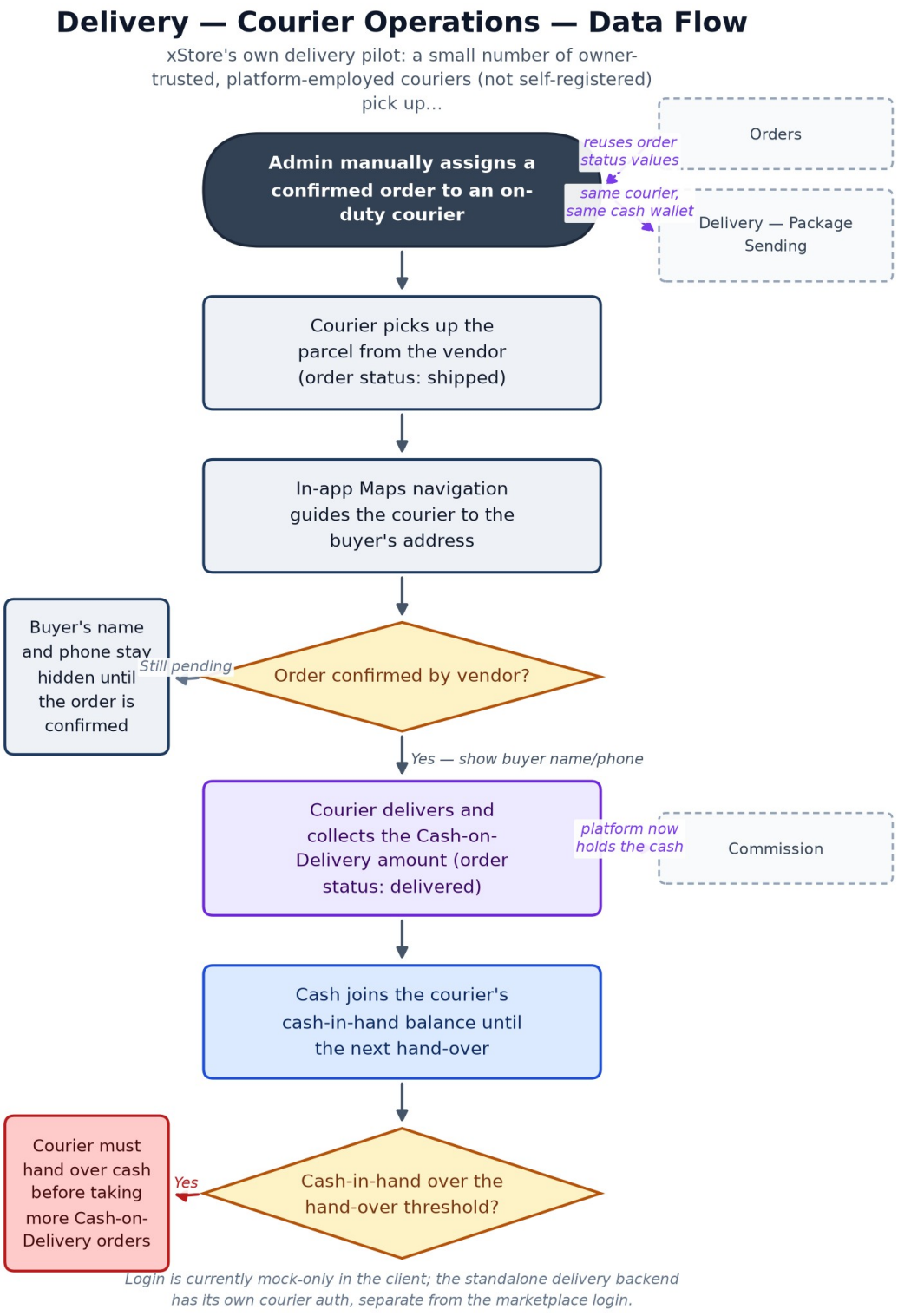
Business Summary

xStore's own delivery pilot: a small number of owner-trusted, platform-employed couriers (not self-registered) pick up confirmed orders from vendors, deliver them, and collect the cash-on-delivery payment directly from the buyer — solving the commission-collection problem structurally, since the platform now holds the cash itself instead of trusting the vendor to pay later.

Why It Matters

This is a genuine strategic choice, not just a feature: it changes who is holding the buyer's cash at the moment of delivery. Piloted deliberately small — one city, 2–3 trusted couriers, manually assigned by an admin — before any wider rollout. It complements, rather than replaces, the vendor self-delivery + owed-wallet model in Commission, which remains the fallback outside covered zones.

Data Flow


Delivery — Courier Operations — Data Flow
xStore's own delivery pilot: a small number of owner-trusted, platform-employed couriers (not self-registered) pick up...
Admin manually assigns a confirmed order to an on-duty courier
reuses order status values
Orders
same courier, same cash wallet
Delivery — Package Sending
Courier picks up the parcel from the vendor (order status: shipped)
In-app Maps navigation guides the courier to the buyer's address
Buyer's name and phone stay hidden until the order is confirmed
Still pending
Order confirmed by vendor?
Yes — show buyer name/phone
Courier delivers and collects the Cash-on-Delivery amount (order status: delivered)
platform now holds the cash
Commission
Cash joins the courier's cash-in-hand balance until the next hand-over
Courier must hand over cash before taking more Cash-on-Delivery orders
Yes
Cash-in-hand over the hand-over threshold?
Login is currently mock-only in the client; the standalone delivery backend has its own courier auth, separate from the marketplace login.

Connects With

Module	Relationship	Direction
Orders & Fulfillment Status	Reuses the existing order-status lifecycle (shipped = picked up, delivered = dropped off) instead of a parallel one.	shares state model with
Commission & Vendor Wallet	Changes the money flow: platform holds the cash directly instead of the vendor owing a fee later.	alternate flow to
Delivery — Package Sending	Same courier pool and the same cash-in-hand ledger serve both marketplace orders and standalone package requests.	shares infrastructure with

Key Points for the Team

- Deliberately piloted small: one city, 2–3 owner-trusted couriers, manual admin assignment, daily cash hand-over — this is a controlled test, not a general rollout, and should be described to stakeholders as such.
- Couriers cannot self-register; accounts are created by the xStore team only. This is an intentional trust control given they will be handling customers' cash.
- Buyer name and phone number are deliberately hidden from the courier until the vendor confirms the order — a privacy safeguard worth highlighting to customer service when buyers ask why a courier didn't call ahead of confirmation.
- No hand-over/settlement flow exists yet in the app — the courier's cash balance is visible, but there's no in-app way to mark it as handed over and reset.
- Runs on a separate standalone delivery backend from the main xStoreEcommerce API — worth engineering awareness that these are two different systems with two different auth schemes.

PART 12

Delivery — Package Sending

Status: Pilot — client-side complete, fully mocked — Full request → price → confirm → pick up → deliver flow is built and test-covered on mock data. No backend integration run yet this cycle.

Location: lib/features/delivery/ (consumer package request)

Backend: Standalone delivery-backend (planned) — /api/delivery-requests/*

Actors: Buyer, Vendor, Courier, Admin

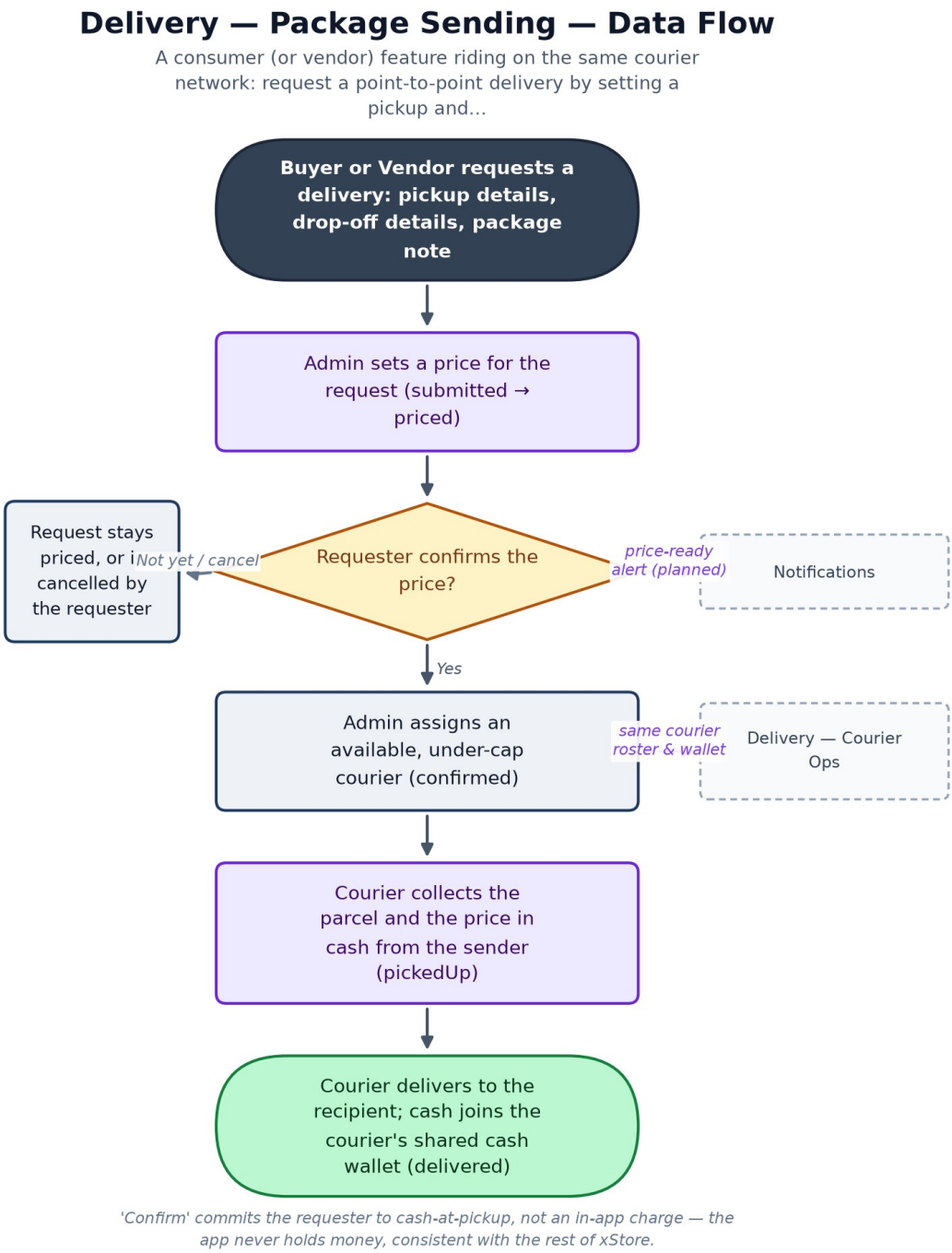
Business Summary

A consumer (or vendor) feature riding on the same courier network: request a point-to-point delivery by setting a pickup and a destination, xStore's admin sets the price, the requester confirms — which commits them to paying the courier in cash at pickup — and an xStore courier collects the parcel and cash and delivers it.

Why It Matters

This extends the courier pilot into a standalone revenue line beyond marketplace orders — effectively a light local-courier service. It deliberately mirrors the Cash-on-Delivery rule everywhere else in the app: xStore never holds the money, so 'paying' currently means cash changes hands at pickup, not an in-app payment. Real in-app prepayment is a separate, larger project requiring a payment gateway that does not exist today — worth stakeholders knowing this distinction explicitly.

Data Flow



Connects With

Module	Relationship	Direction
Delivery — Courier Operations	Uses the exact same courier pool and cash-in-hand ledger as marketplace order delivery.	shares infrastructure with

Module	Relationship	Direction
Notifications	Price-ready and status-change moments are natural alert triggers, not yet wired.	should feed

Key Points for the Team

- Customer-facing name and phone are hidden from the courier until the request is confirmed — same privacy rule as marketplace orders, applied consistently.
- 'Admin sets the price' means there is no live, scalable pricing model yet — every request is priced by a human. Fine at pilot volume, a real constraint if package volume grows.
- Open business decisions not yet made: who eats the cost when a courier goes to pick up a package the sender then refuses to hand over, and what items are prohibited from being shipped this way — worth a decision before wider rollout.
- Entirely mocked end-to-end today; no backend has been connected to this flow yet in this cycle.

PART 13

Wishlist

Status: Partial — Functional against mock or live data. Buyer-only; vendor accounts do not see a wishlist.
Location: lib/features/wishlist/
Backend: xStoreEcommerce API — GET/POST/DELETE /api/wishlist/{id}/*, PUT /api/wishlist/items/{listingId}
Actors: Buyer

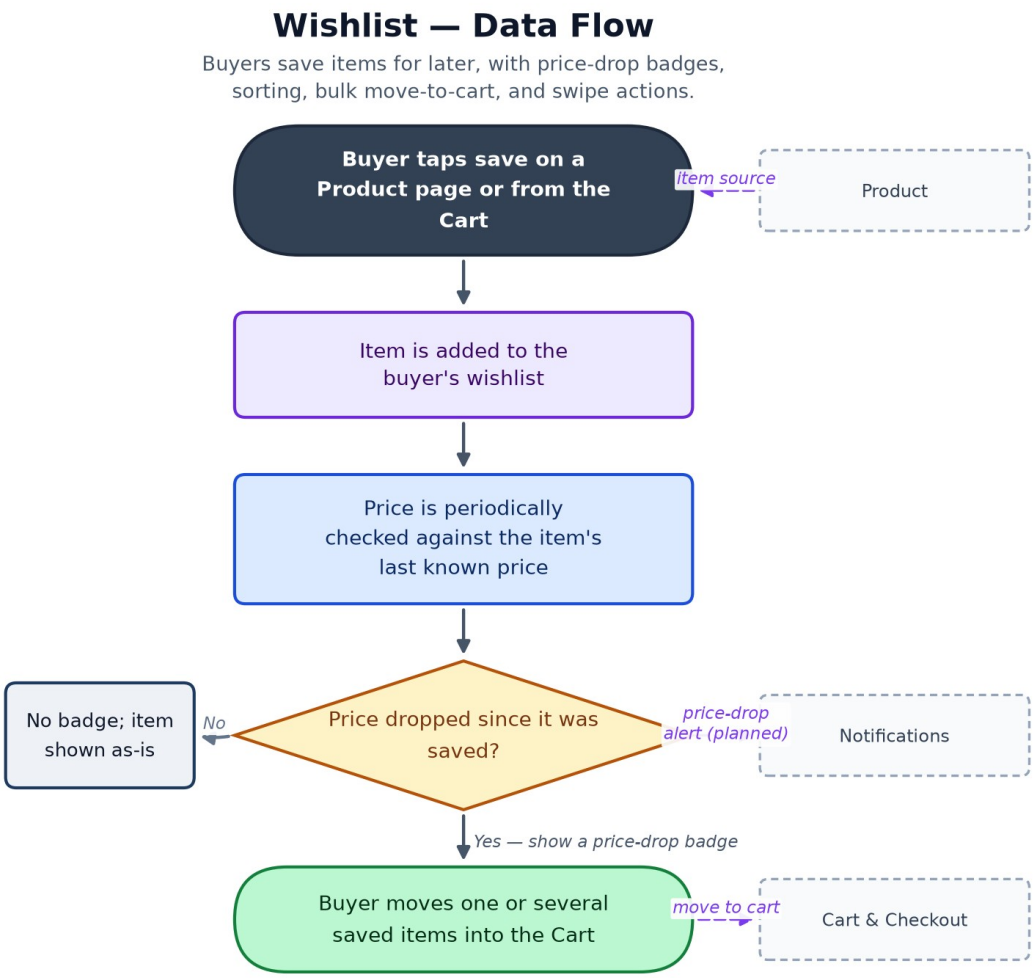
Business Summary

Buyers save items for later, with price-drop badges, sorting, bulk move-to-cart, and swipe actions. A retention tool for people who are interested but not ready to buy today.

Why It Matters

A wishlist add is a soft purchase-intent signal and a natural hook for a 're-engage this buyer' push notification when a saved item drops in price or comes back in stock — but that hook only becomes real once push notifications exist (see Notifications), which is currently the weakest module in the app.

Data Flow



Connects With

Module	Relationship	Direction
Product	Every wishlist entry is a saved product.	reads
Cart & Checkout	Move-to-cart hands items across directly.	feeds
Notifications	Price-drop and back-in-stock are designed to trigger a push alert — not yet wired since push does not exist.	should feed

Key Points for the Team

- The retention value of this module is currently capped by Notifications having no real push infrastructure — a price drop happens silently today unless the buyer happens to reopen the wishlist.

PART 14

Profile & Vendor Store Page

Status: Partial — Functional against mock/live data; avatar upload returns a placeholder image rather than actually uploading.

Location: lib/features/profile/

Backend: xStoreEcommerce API — GET/PUT /api/users/{id}, GET /api/users/{id}/store, POST /api/users/{id}/avatar, DELETE /api/users/me

Actors: Buyer, Vendor

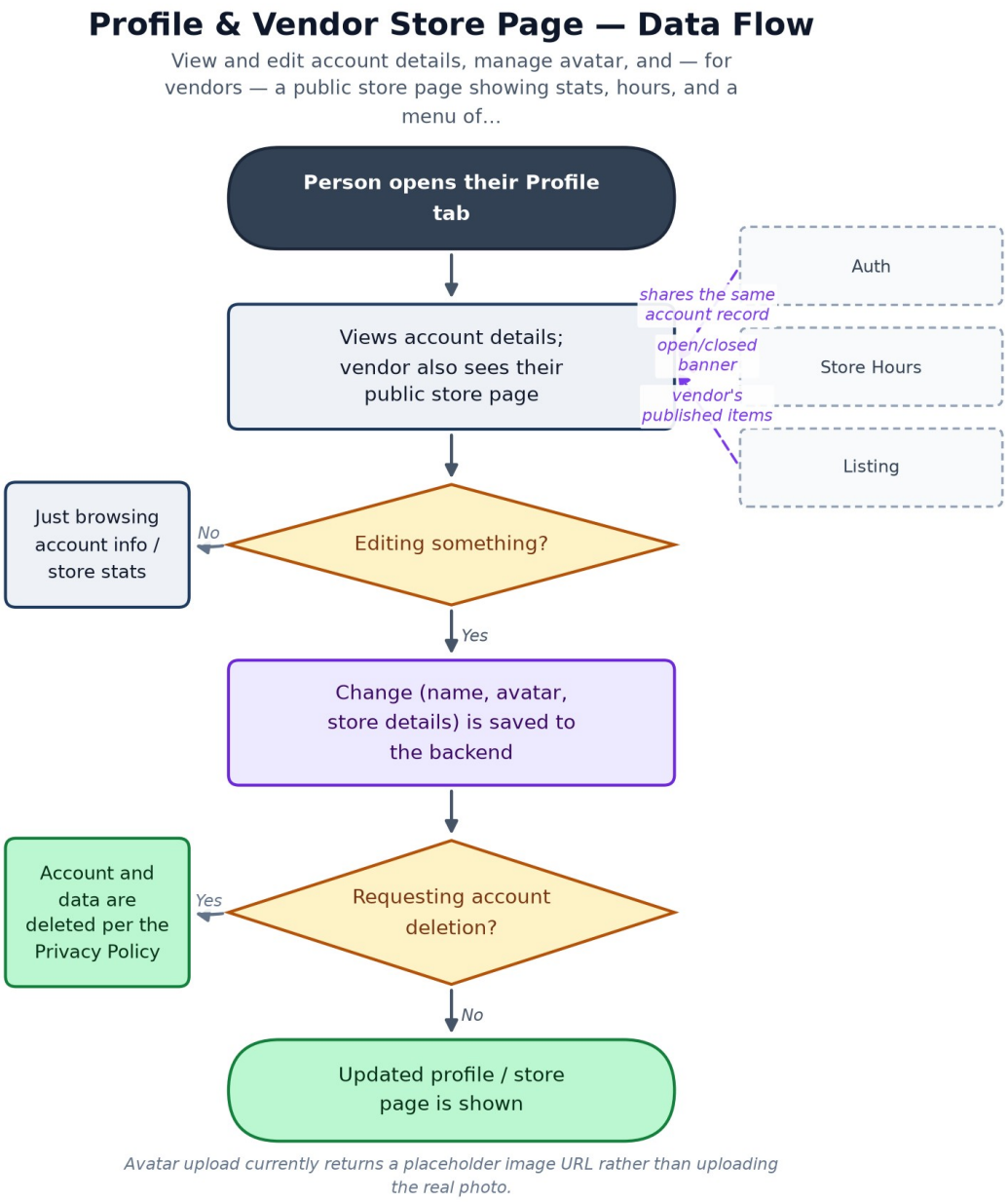
Business Summary

View and edit account details, manage avatar, and — for vendors — a public store page showing stats, hours, and a menu of shortcuts to their listings and orders.

Why It Matters

The vendor store page is part of xStore's public face to buyers, so it belongs on marketing's radar as much as engineering's. On the compliance side, there's good news: a real delete-account endpoint already exists, which is exactly what's needed to honor a user's data-deletion request under the Privacy Policy.

Data Flow



Avatar upload currently returns a placeholder image URL rather than uploading the real photo.

Connects With

Module	Relationship	Direction
Authentication & Identity	Reads and edits the same account record created at sign-up.	shared data
Store Hours	Vendor's store page shows the open/closed banner from this module.	displays
Listing	Vendor's store page links out to their published listings.	displays

Key Points for the Team

- Avatar upload doesn't actually work — it silently returns a placeholder image regardless of the photo picked, which will confuse both buyers and vendors trying to personalize their account.
- Delete-account already exists as a real, working backend call — good foundation for privacy-policy compliance; worth confirming customer service knows this path exists for handling deletion requests.

PART 15

Notifications

Status: Stubbed — Entirely simulated on-device data with artificial delays; zero real endpoints; no push SDK integrated.
Location: lib/features/notifications/
Backend: None — no push service (e.g. Firebase Cloud Messaging) is wired in yet; the in-app inbox is 100% fake data
Actors: Buyer, Vendor

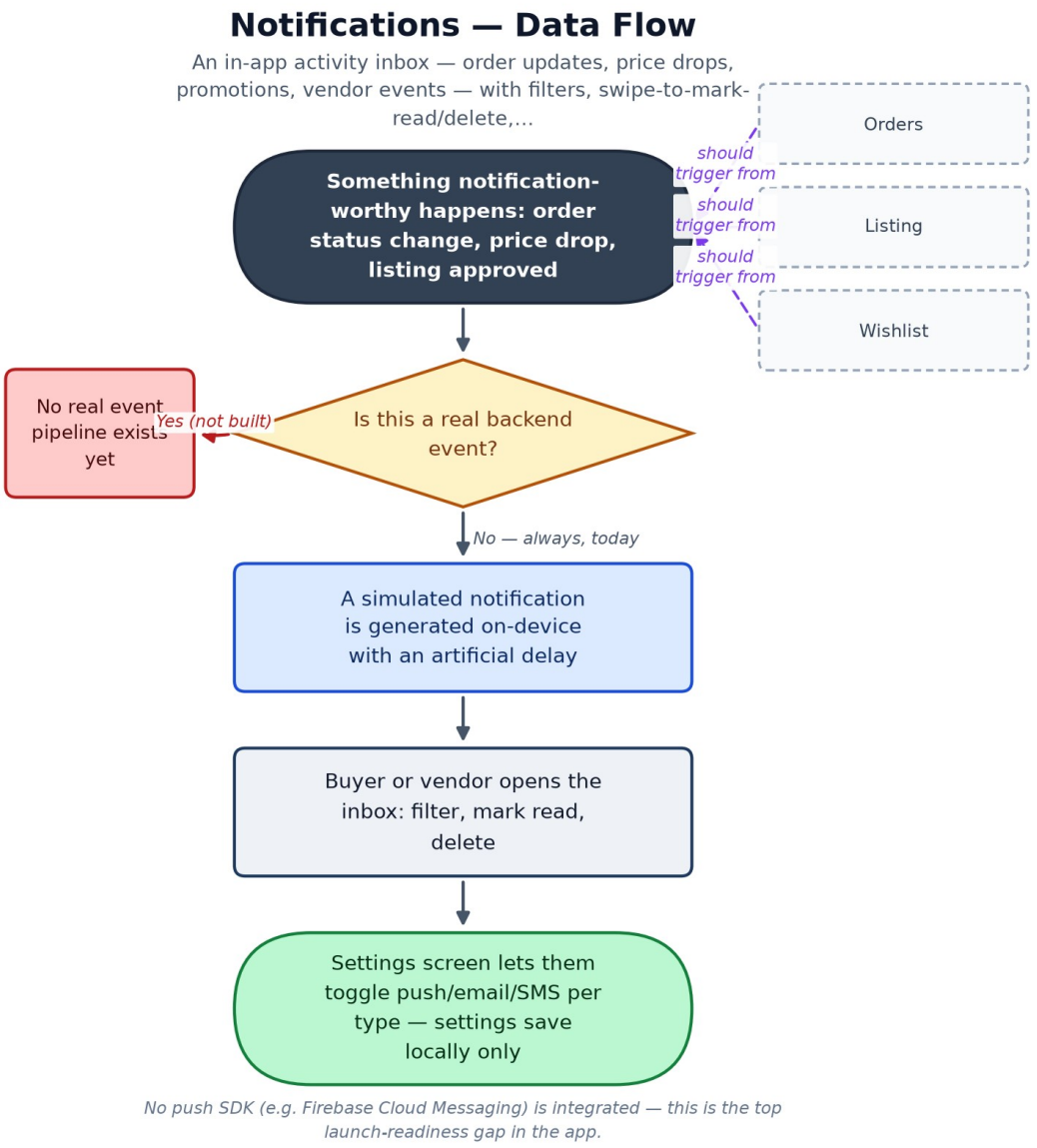
Business Summary

An in-app activity inbox — order updates, price drops, promotions, vendor events — with filters, swipe-to-mark-read/delete, and a settings screen to toggle push/email/SMS per event type.

Why It Matters

This is the weakest module in the app today and a real launch risk: there is no push notification infrastructure at all, so a buyer gets no real-time alert when their order status changes, and everything shown in the inbox is fabricated demo content, not real events. That directly increases customer-service load, since buyers have to open the app and check manually rather than being told when something happens — worth flagging clearly to customer service and stakeholders as a known, current limitation.

Data Flow



Connects With

Module	Relationship	Direction
Orders & Fulfillment Status	Order status changes are the most important intended trigger — not yet wired to anything real.	should consume
Listing	Listing-approval and low-stock events are intended triggers.	should consume
Wishlist	Price-drop and back-in-stock events are intended triggers.	should consume

Key Points for the Team

- Zero real endpoints and no push SDK — this is the top launch-readiness gap in the entire app; every other module's 'notify the user' moment quietly depends on this one being built for real.

- Notification preferences (push/email/SMS toggles) are saved only on the device today, so they would not survive a reinstall or transfer to a new phone.
- Customer-service note: because the inbox shows simulated content, any buyer reporting 'a notification I got doesn't match what happened to my order' is not a bug — the inbox isn't connected to real events yet.

Consolidated Launch-Readiness & Risk Register

Every module Part above flags its own open items under Key Points for the Team. This table rolls all of them up into one prioritized list, so Development, Marketing, Customer Service, and stakeholders can plan against the same picture without re-reading all fifteen Parts.

Severity is a business-impact judgment, not a formal engineering estimate: Critical items block a real, wide-scale launch outright; High items create real cost, confusion, or risk but don't stop a controlled pilot; Medium and Low items are real gaps worth planning for but are not launch blockers.

Severity	Area	Issue
Critical	**Orders**	No live Orders/Checkout backend exists yet. Commission, Courier Operations, Package Sending, and Notifications are all building against mocked order data — this is the single biggest structural dependency in the app.
Critical	**Notifications**	No push notification infrastructure at all (no Firebase Cloud Messaging or equivalent). Buyers and vendors get no real-time alert when an order status changes, a price drops, or a listing is approved.
Critical	**Listing**	Product photo upload does not actually upload — images are stored as local file paths, not real files. Blocks real vendor onboarding.
Critical	**Product**	Product pages show fabricated trust and stock data to real buyers: stock defaults to a flat 99 units, seller rating/sales/verified badge are hardcoded placeholders, not computed from real activity.
Critical	**Explore & Home**	Live-probed 2026-08-06: the hosted backend hard-requires X-Latitude/X-Longitude headers on listing search and the app never sends them — this breaks product search and Home's hot-deals section for every real user today.
High	**Cart & Checkout**	Legacy card-payment fields are collected on the checkout form but never transmitted anywhere. Since xStore is Cash-on-Delivery only, these fields should be removed rather than left as a non-functional, confusing step.
High	**Commission**	Commission rate and both wallet thresholds are hardcoded constants in the app, not configurable from any backend or admin screen — every pricing change requires an app release.
High	**Commission / Courier Operations**	No settlement / hand-over flow exists for either the vendor commission wallet or the courier cash-in-hand balance — both show a live owed amount with no in-app way to mark it paid.
High	**Profile**	Avatar upload silently returns a placeholder image URL regardless of the photo picked — the real photo never uploads.
High	**Authentication**	Live-probed 2026-08-06: /auth/forgot-password 500s (backend SMTP credentials misconfigured) — password reset is completely broken for any user without another recovery path.
High	**Orders**	The payment-method list still includes non-Egyptian options (cibCard,

Severity	Area	Issue
		dahabiCard, baridimob) left over from an earlier market; only Cash-on-Delivery should remain for the Egypt launch.
Medium	**Listing**	No reliable vendorId on the listing record itself — ownership is only inferred from whose session token created it.
Medium	**Listing**	No true variant/SKU model — one price and one stock number per listing; a real constraint for vendors selling size or colour variants.
Medium	**Cart & Checkout**	Cart is an in-memory list while running on mock data and does not survive an app restart; checkout still uses a hardcoded sample address instead of the buyer's saved address.
Medium	**Reference Data**	Listing's create-product screen still uses its own hardcoded category list instead of the shared Reference Data taxonomy.
Medium	**Home**	Live-probed 2026-08-06: GET /api/home 500s reproducibly (backend database-concurrency bug). No current screen depends on it directly, but the same bug signature likely affects other endpoints intermittently.
Medium	**Commission**	Pause enforcement blocks a vendor from publishing new listings when over threshold, but does not retroactively pause their already-live listings.
Medium	**Delivery — Package Sending**	Every delivery price is set manually by an admin — fine at pilot volume, a real constraint if package volume grows.
Medium	**Delivery**	Open business decisions not yet made: who pays the delivery fee, who eats the cost of a refused Cash-on-Delivery parcel or a sender who refuses to hand over a package at pickup.
Low	**Engineering housekeeping**	Several route groups (cart, wishlist, orders, store-hours) are called via inline path strings rather than being registered in the shared ApiEndpoints file — a consistency cleanup, not a functional bug.
Low	**Reference Data**	Governorates are modeled in the API as belonging to a city — the inverse of Egypt's real-world administrative hierarchy; worth confirming intent with the backend team.
Low	**Authentication**	Logout is local-only — the device forgets the session token, but the backend is never told the session ended.

Glossary

COD (Cash on Delivery)	The buyer pays the courier or vendor in cash at the moment of delivery. xStore is a COD-dominant marketplace — the app itself never holds buyer money.
EGP	Egyptian Pound, xStore's operating currency.
Vendor	A seller with a store on xStore. Vendors list products, fulfil orders, and owe the platform a commission on what they sell.
Buyer / Consumer	A shopper on xStore. Buyers browse, order, and pay Cash on Delivery.
Courier	An xStore-employed delivery person (not self-registered) who picks up confirmed orders or package requests, delivers them, and collects the Cash-on-Delivery payment.
Admin	The xStore team, operating through the separate Admin Dashboard web app — assigns couriers, prices package requests, and (in future) manages merchandising and commission config.
Listing	A single product a vendor has published — the base unit of the catalog.
Commission / platform fee	The percentage of a sale xStore keeps, deducted from what the vendor earns (never added to the buyer's price).
Owed-balance wallet	A running total of commission a vendor (or cash a courier) currently owes xStore, settled periodically outside the app today.
Order status lifecycle	pending → confirmed → processing → shipped → delivered, with cancelled / refunded as exit states. Shipped means picked up by a courier or vendor; delivered means the buyer has the item and cash has changed hands.
MOCK mode	A build-time flag the app runs under during development — screens show realistic fake data instead of calling the real backend. Most modules currently run this way some or all of the time.
Bearer token / session token	The credential issued at login that proves who is calling the backend on every subsequent request.
REST endpoint	A specific backend web address (e.g. GET /api/listings) the app calls to read or write one kind of data.
Push notification	A real-time alert sent to a phone even when the app is closed. xStore does not have this infrastructure yet (see Notifications, Part 15).
xStoreEcommerce API	The main .NET marketplace backend: accounts, catalog, listings, orders, users, reference data.
delivery-backend	A separate, standalone .NET service that powers courier operations, package delivery, and the courier cash wallet.

Closing Note

This document reflects the app as built and the business decisions behind it, cross-checked against the current codebase and the latest live-backend probes as of 9 August 2026. It is meant to be a living reference: as modules move from Partial to Complete, as the Orders backend goes live, and as push notifications and the settlement flows get built, this document should be updated alongside them so it stays the one shared picture Development, Marketing, Customer Service, and stakeholders can all work from.